Input File Description

Program: cp.x / CP / Quantum ESPRESSO (version: 7.5)

TABLE OF CONTENTS

INTRODUCTION

```
Input data format: { } = optional, [ ] = it depends, | = or

All quantities whose dimensions are not explicitly specified are in
HARTREE ATOMIC UNITS. Charge is "number" charge (i.e. not multiplied
by e); potentials are in energy units (i.e. they are multiplied by e)

BEWARE: TABS, CRLF, ANY OTHER STRANGE CHARACTER, ARE A SOURCES OF TROUBLE
USE ONLY PLAIN ASCII TEXT FILES (CHECK THE FILE TYPE WITH UNIX COMMAND "file")

Namelists must appear in the order given below.
Comment lines in namelists can be introduced by a "!", exactly as in
fortran code. Comments lines in ``cards'' can be introduced by
either a "!" or a "#" character in the first position of a line.
Do not start any line in ``cards'' with a "/" character.
Leave a space between card names and card options, e.g.
ATOMIC_POSITIONS (bohr), not ATOMIC_POSITIONS(bohr)

Structure of the input data:
================================================================================

&CONTROL
  ...
/

&SYSTEM
 ...
/

&ELECTRONS
...
/

[ &IONS
  ...
 / ]

[ &CELL
  ...
 / ]

[ &WANNIER
  ...
 / ]

ATOMIC_SPECIES
 X  Mass_X  PseudoPot_X
 Y  Mass_Y  PseudoPot_Y
 Z  Mass_Z  PseudoPot_Z

ATOMIC_POSITIONS { alat | bohr | crystal | angstrom }
  X 0.0  0.0  0.0  {if_pos(1) if_pos(2) if_pos(3)}
  Y 0.5  0.0  0.0
  Z 0.0  0.2  0.2

[ CELL_PARAMETERS { alat | bohr | angstrom }
   v1(1) v1(2) v1(3)
   v2(1) v2(2) v2(3)
```

```
   v3(1) v3(2) v3(3) ]

[ OCCUPATIONS
   f_inp1(1)  f_inp1(2)  f_inp1(3) ... f_inp1(10)
   f_inp1(11) f_inp1(12) ... f_inp1(nbnd)
 [ f_inp2(1)  f_inp2(2)  f_inp2(3) ... f_inp2(10)
   f_inp2(11) f_inp2(12) ... f_inp2(nbnd) ] ]

[ CONSTRAINTS
   nconstr  { constr_tol }
   constr_type(.)   constr(1,.)   constr(2,.) [ constr(3,.)   constr(4,.) ] { constr_target(.) } ]

[ ATOMIC_FORCES
   label_1 Fx(1) Fy(1) Fz(1)
   .....
   label_n Fx(n) Fy(n) Fz(n) ]
```

Namelist: &CONTROL

calculation CHARACTER

Default: 'cp'

```
a string describing the task to be performed:
   'cp',
   'scf',
   'nscf',
   'relax',
   'vc-relax',
   'vc-cp',
   'cp-wf',
   'vc-cp-wf'

   (vc = variable-cell).
   (wf = Wannier functions).
```

[Back to Top]

title CHARACTER

Default: 'MD Simulation '

```
reprinted on output.
```

[Back to Top]

verbosity CHARACTER

Default: 'low'

```
In order of decreasing verbose output:
 'debug' | 'high' | 'medium' | 'low','default' | 'minimal'
```

[Back to Top]

isave INTEGER

Default: 100

See: ndr

See: ndw

```
Number of steps between successive savings of
information needed to restart the run.
```

[Back to Top]

restart_mode CHARACTER

Default: 'restart'

```
'from_scratch'   : from scratch
'restart'        : from previous interrupted run
'reset_counters' : continue a previous simulation,
                   performs  "nstep" new steps, resetting
                   the counter and averages
```

[Back to Top]

nstep INTEGER

Default: 50

number of Car-Parrinello steps performed in this run

[Back to Top]

iprint INTEGER

Default: 10

Number of steps between successive writings of relevant physical quantities
to files named as "prefix.???" depending on "prefix" parameter.
In the standard output relevant quantities are written every 10*iprint steps.

[Back to Top]

tstress LOGICAL

Default: .false.

Write stress tensor to standard output each "iprint" steps.
It is set to .TRUE. automatically if calculation='vc-relax'

[Back to Top]

tprnfor LOGICAL

Default: .false.

print forces. Set to .TRUE. when ions are moving.

[Back to Top]

dt REAL

Default: 1.D0

time step for molecular dynamics, in Hartree atomic units
(1 a.u.=2.4189 * 10^-17 s : beware, PW code use
 Rydberg atomic units, twice that much!!!)

[Back to Top]

outdir CHARACTER

Default: value of the ESPRESSO_TMPDIR environment variable if set; current directory ('./') otherwise

input, temporary, trajectories and output files are found
in this directory.

[Back to Top]

saverho LOGICAL

This flag controls the saving of charge density in CP codes:
If .TRUE.   save charge density to restart dir,
If .FALSE. do not save charge density.

[Back to Top]

prefix CHARACTER

Default: 'cp'

prepended to input/output filenames and restart folders:
  prefix.pos : atomic positions
  prefix.vel : atomic velocities
  prefix.for : atomic forces
  prefix.cel : cell parameters
  prefix.str : stress tensors
  prefix.evp : energies
  prefix.hrs : Hirshfeld effective volumes (ts-vdw)
  prefix.eig : eigen values
  prefix.nos : Nose-Hoover variables
  prefix.spr : spread of Wannier orbitals
  prefix.wfc : center of Wannier orbitals

```
  prefix.ncg : number of Poisson CG steps (PBE0)
  prefix_ndw.save/ : write restart folder
  prefix_ndr.save/ : read restart folder
where ndr and ndw are the integers number described below
```

[Back to Top]

ndr INTEGER

Default: 50

```
The restart files are read from the folder
  outdir/prefix_ndr.save/
where outdir, prefix and ndr are the input variables described
in this document
```

[Back to Top]

ndw INTEGER

Default: 50

```
The restart files are written, if ndw > 0, in the folder
  outdir/prefix_ndw.save/
where outdir, prefix and ndw are the input variables described
in this document
```

[Back to Top]

tabps LOGICAL

Default: .false.

```
.true. to compute the volume and/or the surface of an isolated
system for finite pressure/finite surface tension calculations
(PRL 94, 145501 (2005); JCP 124, 074103 (2006)).
```

[Back to Top]

max_seconds REAL

Default: 1.D+7, or 150 days, i.e. no time limit

```
jobs stops after max_seconds CPU time. Used to prevent
a hard kill from the queuing system.
```

[Back to Top]

etot_conv_thr REAL

Default: 1.0D-4

```
convergence threshold on total energy (a.u) for ionic
minimization: the convergence criterion is satisfied
when the total energy changes less than etot_conv_thr
between two consecutive scf steps.
See also forc_conv_thr - both criteria must be satisfied
```

[Back to Top]

forc_conv_thr REAL

Default: 1.0D-3

```
convergence threshold on forces (a.u) for ionic
minimization: the convergence criterion is satisfied
when all components of all forces are smaller than
forc_conv_thr.
See also etot_conv_thr - both criteria must be satisfied
```

[Back to Top]

ekin_conv_thr REAL

Default: 1.0D-6

```
convergence criterion for electron minimization:
convergence is achieved when "ekin < ekin_conv_thr".
See also etot_conv_thr - both criteria must be satisfied.
```

[Back to Top]

disk_io CHARACTER

Default: 'default'

```
'high': CP code will write Kohn-Sham wfc files and additional
        information in data-file.xml in order to restart
        with a PW calculation or to use postprocessing tools.
        If disk_io is not set to 'high', the data file
        written by CP will not be readable by PW or PostProc.
```

[Back to Top]

memory CHARACTER

Default: 'default'

```
'small': NO LONGER IMPLEMENTED SINCE v.6.3
         memory-saving tricks are implemented. Currently:
         - the G-vectors are sorted only locally, not globally
         - they are not collected and written to file
         For large systems, the memory and time gain is sizable
         but the resulting data files are not portable - use it
         only if you do not need to re-read the data file
```

[Back to Top]

pseudo_dir CHARACTER

Default: value of the $ESPRESSO_PSEUDO environment variable if set; '$HOME/espresso/pseudo/' otherwise

```
directory containing pseudopotential files
```

[Back to Top]

tefield LOGICAL

Default: .FALSE.

```
If .TRUE. a homogeneous finite electric field described
through the modern theory of the polarization is applied.
```

[Back to Top]

Namelist: &SYSTEM

ibrav INTEGER

Status: REQUIRED

```
  Bravais-lattice index. If ibrav /= 0, specify EITHER
  [ celldm(1)-celldm(6) ] OR [ A,B,C,cosAB,cosAC,cosBC ]
  but NOT both. The lattice parameter "alat" is set to
  alat = celldm(1) (in a.u.) or alat = A (in Angstrom);
  see below for the other parameters.
  For ibrav=0 specify the lattice vectors in CELL_PARAMETER,
  optionally the lattice parameter alat = celldm(1) (in a.u.)
  or = A (in Angstrom), or else it is taken from CELL_PARAMETERS

ibrav      structure                   celldm(2)-celldm(6)
                                     or: b,c,cosbc,cosac,cosab
  0          free
      crystal axis provided in input: see card CELL_PARAMETERS

  1          cubic P (sc)
      v1 = a(1,0,0),  v2 = a(0,1,0),  v3 = a(0,0,1)

  2          cubic F (fcc)
      v1 = (a/2)(-1,0,1),  v2 = (a/2)(0,1,1), v3 = (a/2)(-1,1,0)

  3          cubic I (bcc)
      v1 = (a/2)(1,1,1),  v2 = (a/2)(-1,1,1),  v3 = (a/2)(-1,-1,1)
 -3          cubic I (bcc), more symmetric axis:
      v1 = (a/2)(-1,1,1), v2 = (a/2)(1,-1,1),  v3 = (a/2)(1,1,-1)
```

```
 4          Hexagonal and Trigonal P        celldm(3)=c/a
     v1 = a(1,0,0),  v2 = a(-1/2,sqrt(3)/2,0),  v3 = a(0,0,c/a)

 5          Trigonal R, 3fold axis c        celldm(4)=cos(gamma)
     The crystallographic vectors form a three-fold star around
     the z-axis, the primitive cell is a simple rhombohedron:
     v1 = a(tx,-ty,tz),   v2 = a(0,2ty,tz),   v3 = a(-tx,-ty,tz)
     where c=cos(gamma) is the cosine of the angle gamma between
     any pair of crystallographic vectors, tx, ty, tz are:
       tx=sqrt((1-c)/2), ty=sqrt((1-c)/6), tz=sqrt((1+2c)/3)
-5          Trigonal R, 3fold axis <111>    celldm(4)=cos(gamma)
     The crystallographic vectors form a three-fold star around
     <111>. Defining a' = a/sqrt(3) :
     v1 = a' (u,v,v),   v2 = a' (v,u,v),   v3 = a' (v,v,u)
     where u and v are defined as
       u = tz - 2*sqrt(2)*ty,  v = tz + sqrt(2)*ty
     and tx, ty, tz as for case ibrav=5
     Note: if you prefer x,y,z as axis in the cubic limit,
           set  u = tz + 2*sqrt(2)*ty,  v = tz - sqrt(2)*ty
           See also the note in Modules/latgen.f90

 6          Tetragonal P (st)               celldm(3)=c/a
     v1 = a(1,0,0),  v2 = a(0,1,0),  v3 = a(0,0,c/a)

 7          Tetragonal I (bct)              celldm(3)=c/a
     v1=(a/2)(1,-1,c/a),  v2=(a/2)(1,1,c/a),  v3=(a/2)(-1,-1,c/a)

 8          Orthorhombic P                  celldm(2)=b/a
                                            celldm(3)=c/a
     v1 = (a,0,0),  v2 = (0,b,0), v3 = (0,0,c)

 9          Orthorhombic base-centered(bco) celldm(2)=b/a
                                            celldm(3)=c/a
     v1 = (a/2, b/2,0),  v2 = (-a/2,b/2,0),  v3 = (0,0,c)
-9          as 9, alternate description
     v1 = (a/2,-b/2,0),  v2 = (a/2, b/2,0),  v3 = (0,0,c)

 10         Orthorhombic face-centered      celldm(2)=b/a
                                            celldm(3)=c/a
     v1 = (a/2,0,c/2),  v2 = (a/2,b/2,0),  v3 = (0,b/2,c/2)

 11         Orthorhombic body-centered      celldm(2)=b/a
                                            celldm(3)=c/a
     v1=(a/2,b/2,c/2),  v2=(-a/2,b/2,c/2),  v3=(-a/2,-b/2,c/2)

 12         Monoclinic P, unique axis c     celldm(2)=b/a
                                            celldm(3)=c/a,
                                            celldm(4)=cos(ab)
     v1=(a,0,0), v2=(b*cos(gamma),b*sin(gamma),0),  v3 = (0,0,c)
     where gamma is the angle between axis a and b.
-12         Monoclinic P, unique axis b     celldm(2)=b/a
                                            celldm(3)=c/a,
                                            celldm(5)=cos(ac)
     v1 = (a,0,0), v2 = (0,b,0), v3 = (c*cos(beta),0,c*sin(beta))
     where beta is the angle between axis a and c

 13         Monoclinic base-centered        celldm(2)=b/a
                                            celldm(3)=c/a,
                                            celldm(4)=cos(gamma)
     v1 = (  a/2,         0,                -c/2),
     v2 = (b*cos(gamma), b*sin(gamma),       0  ),
     v3 = (  a/2,         0,                 c/2),
     where gamma=angle between axis a and b projected on xy plane

-13         Monoclinic base-centered        celldm(2)=b/a
             (unique axis b)                celldm(3)=c/a,
                                            celldm(5)=cos(beta)
     v1 = (  a/2,       b/2,             0),
     v2 = ( -a/2,       b/2,             0),
     v3 = (c*cos(beta),   0,   c*sin(beta)),
     where beta=angle between axis a and c projected on xz plane
 IMPORTANT NOTICE: until QE v.6.4.1, axis for ibrav=-13 had a
 different definition: v1(old) = v2(now), v2(old) = -v1(now)

 14         Triclinic                       celldm(2)= b/a,
```

```
                                          celldm(3)= c/a,
                                          celldm(4)= cos(bc),
                                          celldm(5)= cos(ac),
                                          celldm(6)= cos(ab)
        v1 = (a, 0, 0),
        v2 = (b*cos(gamma), b*sin(gamma), 0)
        v3 = (c*cos(beta),  c*(cos(alpha)-cos(beta)cos(gamma))/sin(gamma),
             c*sqrt( 1 + 2*cos(alpha)cos(beta)cos(gamma)
                       - cos(alpha)^2-cos(beta)^2-cos(gamma)^2 )/sin(gamma) )
     where alpha is the angle between axis b and c
           beta is the angle between axis a and c
          gamma is the angle between axis a and b
```

[Back to Top]

Either:

celldm(i), i=1,6 REAL

See: ibrav

```
Crystallographic constants - see the "ibrav" variable.
Specify either these OR A,B,C,cosAB,cosBC,cosAC NOT both.
Only needed values (depending on "ibrav") must be specified
alat = celldm(1) is the lattice parameter "a" (in BOHR)
If ibrav=0, only celldm(1) is used if present;
cell vectors are read from card CELL_PARAMETERS
```

[Back to Top]

Or:

A, B, C, cosAB, cosAC, cosBC REAL

```
Traditional crystallographic constants: a,b,c in ANGSTROM
  cosAB = cosine of the angle between axis a and b (gamma)
  cosAC = cosine of the angle between axis a and c (beta)
  cosBC = cosine of the angle between axis b and c (alpha)
The axis are chosen according to the value of "ibrav".
Specify either these OR "celldm" but NOT both.
Only needed values (depending on "ibrav") must be specified
The lattice parameter alat = A (in ANGSTROM )
If ibrav = 0, only A is used if present;
cell vectors are read from card CELL_PARAMETERS
```

[Back to Top]

nat INTEGER

Status: REQUIRED

```
number of atoms in the unit cell
```

[Back to Top]

ntyp INTEGER

Status: REQUIRED

```
number of types of atoms in the unit cell
```

[Back to Top]

nbnd INTEGER

Default: for an insulator, nbnd = number of valence bands (nbnd = # of electrons /2); for a metal, 20% more (minimum 4 more)

```
number of electronic states (bands) to be calculated.
Note that in spin-polarized calculations the number of
k-point, not the number of bands per k-point, is doubled
```

[Back to Top]

tot_charge REAL

Default: 0.0

```
total charge of the system. Useful for simulations with charged cells.
By default the unit cell is assumed to be neutral (tot_charge=0).
tot_charge=+1 means one electron missing from the system,
tot_charge=-1 means one additional electron, and so on.

In a periodic calculation a compensating jellium background is
inserted to remove divergences if the cell is not neutral.
```

[Back to Top]

tot_magnetization REAL

Default: -1 [unspecified]

```
total majority spin charge - minority spin charge.
Used to impose a specific total electronic magnetization.
If unspecified, the tot_magnetization variable is ignored
and the electronic magnetization is determined by the
occupation numbers (see card OCCUPATIONS) read from input.
```

[Back to Top]

ecutwfc REAL

Status: REQUIRED

```
kinetic energy cutoff (Ry) for wavefunctions
```

[Back to Top]

ecutrho REAL

Default: 4 * ecutwfc

```
kinetic energy cutoff (Ry) for charge density and potential
For norm-conserving pseudopotential you should stick to the
default value, you can reduce it by a little but it will
introduce noise especially on forces and stress.
If there are ultrasoft PP, a larger value than the default is
often desirable (ecutrho = 8 to 12 times ecutwfc, typically).
PAW datasets can often be used at 4*ecutwfc, but it depends
on the shape of augmentation charge: testing is mandatory.
The use of gradient-corrected functional, especially in cells
with vacuum, or for pseudopotential without non-linear core
correction, usually requires an higher values of ecutrho
to be accurately converged.
```

[Back to Top]

nr1, nr2, nr3 INTEGER

See: ecutrho

```
three-dimensional FFT mesh (hard grid) for charge
density (and scf potential). If not specified
the grid is calculated based on the cutoff for
charge density.
```

[Back to Top]

nr1s, nr2s, nr3s INTEGER

```
three-dimensional mesh for wavefunction FFT and for the smooth
part of charge density ( smooth grid ).
Coincides with nr1, nr2, nr3 if ecutrho = 4 * ecutwfc ( default )
```

[Back to Top]

nr1b, nr2b, nr3b INTEGER

```
dimensions of the "box" grid for Ultrasoft pseudopotentials
must be specified if Ultrasoft PP are present
```

[Back to Top]

occupations CHARACTER

```
a string describing the occupation of the electronic states.
Allowed values are 'fixed' (default) and 'ensemble'.
In the case of conjugate gradient style of minimization
of the electronic states, if occupations is set to 'ensemble',
this allows ensemble DFT calculations for metallic systems.
```

[Back to Top]

degauss REAL

Default: 0.D0 Ha

```
parameter for the smearing function, only used for ensemble DFT
calculations. Hartree atomic units
```

[Back to Top]

smearing CHARACTER

```
a string describing the kind of occupations for electronic states
in the case of ensemble DFT (occupations == 'ensemble' );
possible values are: 'gaussian', 'fermi-dirac', 'hermite-delta',
'gaussian-splines', 'cold-smearing', 'marzari-vanderbilt', '0', '-1'.
Warning: only 'gaussian' is tested.
```

[Back to Top]

nspin INTEGER

Default: 1

```
nspin = 1 :  non-polarized calculation (default)

nspin = 2 :  spin-polarized calculation, LSDA
             (magnetization along z axis)
```

[Back to Top]

ecfixed REAL

Default: 0.0

See: q2sigma

[Back to Top]

qcutz REAL

Default: 0.0

See: q2sigma

[Back to Top]

q2sigma REAL

Default: 0.1

```
ecfixed, qcutz, q2sigma:  parameters for modified functional to be
used in variable-cell molecular dynamics (or in stress calculation).
"ecfixed" is the value (in Rydberg) of the constant-cutoff;
"qcutz" and "q2sigma" are the height and the width (in Rydberg)
of the energy step for reciprocal vectors whose square modulus
is greater than "ecfixed". In the kinetic energy, G^2 is
replaced by G^2 + qcutz * (1 + erf ( (G^2 - ecfixed)/q2sigma) )
See: M. Bernasconi et al, J. Phys. Chem. Solids 56, 501 (1995)
```

[Back to Top]

input_dft CHARACTER

Default: read from pseudopotential files

```
Exchange-correlation functional: eg 'PBE', 'BLYP' etc
See Modules/funct.f90 for allowed values.
Overrides the value read from pseudopotential files.
Use with care and if you know what you are doing!

Use 'PBE0' to perform hybrid functional calculation using Wannier functions.
Allowed calculation: 'cp-wf' and 'vc-cp-wf'
See CP specific user manual for further guidance (or in CPV/Doc/user_guide.tex)
```

```
and examples in CPV/examples/EXX-wf-example.
Also see related keywords starting with exx_.
```

[Back to Top]

exx_fraction REAL

Default: it depends on the specified functional

```
Fraction of EXX for hybrid functional calculations. In the case of
input_dft='PBE0', the default value is 0.25. This entry overrides
the default (as well as the restart file) value of a given functional.
```

[Back to Top]

lda_plus_u LOGICAL

Default: .FALSE.

```
lda_plus_u = .TRUE. enables calculation with LDA+U
                  ("rotationally invariant"). See also Hubbard_U.
                  Anisimov, Zaanen, and Andersen, PRB 44, 943 (1991);
                  Anisimov et al., PRB 48, 16929 (1993);
                  Liechtenstein, Anisimov, and Zaanen, PRB 52, R5467 (1994);
                  Cococcioni and de Gironcoli, PRB 71, 035105 (2005).
```

[Back to Top]

Hubbard_U(i), i=1,ntyp REAL

Default: 0.D0 for all species

Status: LDA+U works only for a few selected elements. Modify CPV/ldaU.f90 if you plan to use LDA+U with an element that is not configured there.

```
Hubbard_U(i): parameter U (in eV) for LDA+U calculations.
Currently only the simpler, one-parameter LDA+U is
implemented (no "alpha" or "J" terms)
```

[Back to Top]

vdw_corr CHARACTER

Default: 'none'

```
Type of Van der Waals correction. Allowed values:

   'grimme-d2', 'Grimme-D2', 'DFT-D', 'dft-d': semiempirical Grimme's DFT-D2.
    Optional variables: "london_s6", "london_rcut"
    S. Grimme, J. Comp. Chem. 27, 1787 (2006),
    V. Barone et al., J. Comp. Chem. 30, 934 (2009).

    'TS', 'ts', 'ts-vdw', 'ts-vdW', 'tkatchenko-scheffler': Tkatchenko-Scheffler
     dispersion corrections with first-principle derived C6 coefficients
     Optional variables: "ts_vdw_econv_thr", "ts_vdw_isolated"
     See A. Tkatchenko and M. Scheffler, Phys. Rev. Lett. 102, 073005 (2009)
     J. Hermann et al., J. Chem. Phys. 159, 174802 (2023), doi:10.1063/5.0170972

    'XDM', 'xdm': Exchange-hole dipole-moment model. Optional variables: "xdm_a1", "xdm_a2"
     (implemented in PW only)
     A. D. Becke and E. R. Johnson, J. Chem. Phys. 127, 154108 (2007)
      A. Otero de la Roza, E. R. Johnson, J. Chem. Phys. 136, 174109 (2012)

Note that non-local functionals (eg vdw-DF) are NOT specified here but in "input_dft"
```

[Back to Top]

london_s6 REAL

Default: 0.75

```
global scaling parameter for DFT-D. Default is good for PBE.
```

[Back to Top]

london_rcut REAL

Default: 200

```
cutoff radius (a.u.) for dispersion interactions
```

[Back to Top]

ts_vdw LOGICAL

Default: .FALSE.

```
OBSOLESCENT, same as vdw_corr='TS'
```

[Back to Top]

ts_vdw_econv_thr REAL

Default: 1.D-6

```
Optional: controls the convergence of the vdW energy (and forces). The default value
is a safe choice, likely too safe, but you do not gain much in increasing it
```

[Back to Top]

ts_vdw_isolated LOGICAL

Default: .FALSE.

```
Optional: set it to .TRUE. when computing the Tkatchenko-Scheffler vdW energy
for an isolated (non-periodic) system.
```

[Back to Top]

assume_isolated CHARACTER

Default: 'none'

```
  Used to perform calculation assuming the system to be
  isolated (a molecule of a clustr in a 3D supercell).

  Currently available choices:

  'none' (default): regular periodic calculation w/o any correction.

  'makov-payne', 'm-p', 'mp' : the Makov-Payne correction to the
           total energy is computed.
           Theory:
           G.Makov, and M.C.Payne,
           "Periodic boundary conditions in ab initio
           calculations" , Phys.Rev.B 51, 4014 (1995)


var nextffield -type INTEGER {
  default { 0 }
  info {
      Number of activated external ionic force fields.
      See Doc/ExternalForceFields.tex for further explanation and parameterizations
  }
}
```

[Back to Top]

Namelist: &ELECTRONS

electron_maxstep INTEGER

Default: 100

```
maximum number of iterations in a scf step
```

[Back to Top]

electron_dynamics CHARACTER

Default: 'none'

```
set how electrons should be moved
'none'    : electronic degrees of freedom (d.o.f.) are kept fixed
'sd'      : steepest descent algorithm is used to minimize
```

```
            electronic d.o.f.
    'damp'  : damped dynamics is used to propagate electronic d.o.f.
    'verlet'  : standard Verlet algorithm is used to propagate
            electronic d.o.f.
    'cg'    : conjugate gradient is used to converge the
            wavefunction at each ionic step. 'cg' can be used
            interchangeably with 'verlet' for a couple of ionic
            steps in order to "cool down" the electrons and
            return them back to the Born-Oppenheimer surface.
            Then 'verlet' can be restarted again. This procedure
            is useful when electronic adiabaticity in CP is lost
            yet the ionic velocities need to be preserved.
```

[Back to Top]

conv_thr REAL

Default: 1.D-6

```
Convergence threshold for selfconsistency:
estimated energy error < conv_thr
```

[Back to Top]

niter_cg_restart INTEGER

Default: 20

```
frequency in iterations for which the conjugate-gradient algorithm
for electronic relaxation is restarted
```

[Back to Top]

efield REAL

Default: 0.D0

```
Amplitude of the finite electric field (in a.u.;
1 a.u. = 51.4220632*10^10 V/m). Used only if tefield=.TRUE.
```

[Back to Top]

epol INTEGER

Default: 3

```
direction of the finite electric field (only if tefield == .TRUE.)
In the case of a PARALLEL calculation only the case epol==3
is implemented
```

[Back to Top]

emass REAL

Default: 400.D0

```
effective electron mass in the CP Lagrangian, in atomic units
( 1 a.u. of mass = 1/1822.9 a.m.u. = 9.10939 * 10^-31 kg )
```

[Back to Top]

emass_cutoff REAL

Default: 2.5D0

```
mass cut-off (in Rydberg) for the Fourier acceleration
effective mass is rescaled for "G" vector components with
kinetic energy above "emass_cutoff"
```

[Back to Top]

orthogonalization CHARACTER

Default: 'ortho'

```
selects the orthonormalization method for electronic wave
functions
'ortho'       : use iterative algorithm - if it doesn't converge,
                reduce the timestep, or use options ortho_max
                and ortho_eps, or use Gram-Schmidt instead just
```

```
                to start the simulation
'Gram-Schmidt' : use Gram-Schmidt algorithm - to be used ONLY in
                the first few steps.
                YIELDS INCORRECT ENERGIES AND EIGENVALUES.
```

[Back to Top]

ortho_eps REAL

Default: 1.D-8

```
tolerance for iterative orthonormalization
meaningful only if orthogonalization = 'ortho'
```

[Back to Top]

ortho_max INTEGER

Default: 300

```
maximum number of iterations for orthonormalization
meaningful only if orthogonalization = 'ortho'
```

[Back to Top]

ortho_para INTEGER

Default: 0

Status: OBSOLETE: use command-line option " -nd XX" instead

[Back to Top]

electron_damping REAL

Default: 0.1D0

```
damping frequency times delta t, optimal values could be
calculated with the formula :
        SQRT( 0.5 * LOG( ( E1 - E2 ) / ( E2 - E3 ) ) )
where E1, E2, E3 are successive values of the DFT total energy
in a steepest descent simulations.
meaningful only if " electron_dynamics = 'damp' "
```

[Back to Top]

electron_velocities CHARACTER

```
'zero'      : restart setting electronic velocities to zero
'default'   : restart using electronic velocities of the
              previous run
'change_step' : restart simulation using electronic velocities of the
              previous run, with rescaling due to the timestep change.
              specify the old step via tolp as in
              tolp = 'old_time_step_value' in au.
              Note that you may want to specify
              ion_velocities = 'change_step'
```

[Back to Top]

electron_temperature CHARACTER

Default: 'not_controlled'

```
'nose'            : control electronic temperature using Nose
                    thermostat. See also "fnosee" and "ekincw".
'rescaling'       : control electronic temperature via velocities
                    rescaling.
'not_controlled'  : electronic temperature is not controlled.
```

[Back to Top]

ekincw REAL

Default: 0.001D0

```
value of the average kinetic energy (in atomic units) forced
by the temperature control
```

meaningful only with " electron_temperature /= 'not_controlled' "

[Back to Top]

fnosee REAL

Default: 1.D0

oscillation frequency of the nose thermostat (in terahertz)
meaningful only with " electron_temperature = 'nose' "

[Back to Top]

startingwfc CHARACTER

Default: 'random'

'atomic': start from superposition of atomic orbitals
(not yet implemented)

'random': start from random wfcs. See "ampre".

[Back to Top]

tcg LOGICAL

Default: .FALSE.

if .TRUE. perform a conjugate gradient minimization of the
electronic states for every ionic step.
It requires Gram-Schmidt orthogonalization of the electronic
states.

[Back to Top]

maxiter INTEGER

Default: 100

maximum number of conjugate gradient iterations for
conjugate gradient minimizations of electronic states

[Back to Top]

passop REAL

Default: 0.3D0

small step used in the conjugate gradient minimization
of the electronic states.

[Back to Top]

pre_state LOGICAL

Default: .FALSE.

if .TRUE. perform the precondition of the CG gradient
using the kinetic energy of the state.

[Back to Top]

n_inner INTEGER

Default: 2

number of internal cycles for every conjugate gradient
iteration only for ensemble DFT

[Back to Top]

niter_cold_restart INTEGER

Default: 1

frequency in iterations at which a full inner cycle, only
for cold smearing, is performed

[Back to Top]

lambda_cold REAL

Default: 0.03D0

```
step for inner cycle with cold smearing, used when a not full
cycle is performed
```

[Back to Top]

grease REAL

Default: 1.D0

```
a number <= 1, very close to 1: the damping in electronic
damped dynamics is multiplied at each time step by "grease"
(avoids overdamping close to convergence: Obsolete ?)
grease = 1 : normal damped dynamics
```

[Back to Top]

ampre REAL

Default: 0.D0

```
amplitude of the randomization ( allowed values: 0.0 - 1.0 )
meaningful only if " startingwfc = 'random' "
```

[Back to Top]

Namelist: &IONS

input this namelist only if calculation = 'cp', 'relax', 'vc-relax', 'vc-cp', 'cp-wf', 'vc-cp-wf'

ion_dynamics CHARACTER

```
Specify the type of ionic dynamics.

For constrained dynamics or constrained optimisations add the
CONSTRAINTS card (when the card is present the SHAKE algorithm is
                 automatically used).
'none'    : ions are kept fixed
'sd'      : steepest descent algorithm is used to minimize ionic
            configuration
'cg'      : conjugate gradient algorithm is used to minimize ionic
            configuration
'damp'    : damped dynamics is used to propagate ions
'verlet'  : standard Verlet algorithm is used to propagate ions
```

[Back to Top]

ion_positions CHARACTER

Default: 'default'

```
'default '  : if restarting, use atomic positions read from the
              restart file; in all other cases, use atomic
              positions from standard input.

'from_input' : restart the simulation with atomic positions read
              from standard input, even if restarting.
```

[Back to Top]

ion_velocities CHARACTER

Default: 'default'

See: tempw

```
initial ionic velocities
'default'     : restart the simulation with atomic velocities read
                from the restart file
'change_step' : restart the simulation with atomic velocities read
                from the restart file, with rescaling due to the
                timestep change, specify the old step via tolp
```

```
                  as in tolp = 'old_time_step_value' in au.
                  Note that you may want to specify
                  electron_velocities = 'change_step'
    'random'      : start the simulation with random atomic velocities
                  (see also variable tempw)
    'from_input'  : restart the simulation with atomic velocities read
                  from standard input - see card 'ATOMIC_VELOCITIES'
                  BEWARE: tested only with electrons_dynamics='cg'
    'zero'        : restart the simulation with atomic velocities set
                  to zero
```

[Back to Top]

ion_damping REAL

Default: 0.2D0

```
damping frequency times delta t, optimal values could be
  calculated with the formula :
  SQRT( 0.5 * LOG( ( E1 - E2 ) / ( E2 - E3 ) ) )
  where E1, E2, E3 are successive values of the DFT total energy
  in a steepest descent simulations.
  meaningful only if " ion_dynamics = 'damp' "
```

[Back to Top]

ion_radius(i), i=1,ntyp REAL

Default: 0.5 a.u. for all species

```
ion_radius(i): pseudo-atomic radius of the i-th atomic species
used in Ewald summation. Typical values: between 0.5 and 2.
Results should NOT depend upon such parameters if their values
are properly chosen. See also "iesr".
```

[Back to Top]

iesr INTEGER

Default: 1

```
The real-space contribution to the Ewald summation is performed
on iesr*iesr*iesr cells. Typically iesr=1 is sufficient to have
converged results.
```

[Back to Top]

ion_nstepe INTEGER

Default: 1

```
number of electronic steps per ionic step.
```

[Back to Top]

remove_rigid_rot LOGICAL

Default: .FALSE.

```
This keyword is useful when simulating the dynamics and/or the
thermodynamics of an isolated system. If set to true the total
torque of the internal forces is set to zero by adding new forces
that compensate the spurious interaction with the periodic
images. This allows for the use of smaller supercells.

BEWARE: since the potential energy is no longer consistent with
the forces (it still contains the spurious interaction with the
repeated images), the total energy is not conserved anymore.
However the dynamical and thermodynamical properties should be
in closer agreement with those of an isolated system.
Also the final energy of a structural relaxation will be higher,
but the relaxation itself should be faster.
```

[Back to Top]

ion_temperature CHARACTER

Default: 'not_controlled'

```
'nose'          : control ionic temperature using Nose-Hoover
                  thermostat  see parameters "fnosep", "tempw",
                  "nhpcl", "ndega", "nhptyp"
'rescaling'     : control ionic temperature via velocities
                  rescaling. see parameter "tolp"
'not_controlled' : ionic temperature is not controlled
```

[Back to Top]

tempw REAL

Default: 300.D0

```
value of the ionic temperature (in Kelvin) forced by the
temperature control.
meaningful only with " ion_temperature /= 'not_controlled' "
or when the initial velocities are set to 'random'
"ndega" controls number of degrees of freedom used in
temperature calculation
```

[Back to Top]

fnosep REAL

Default: 1.D0

```
oscillation frequency of the nose thermostat (in terahertz)
[note that 3 terahertz = 100 cm^-1]
meaningful only with " ion_temperature = 'nose' "
for Nose-Hoover chain one can set frequencies of all thermostats
( fnosep = X Y Z etc. ) If only first is set, the defaults for
the others will be same.
```

[Back to Top]

tolp REAL

Default: 100.D0

```
tolerance (in Kelvin) of the rescaling. When ionic temperature
differs from "tempw" more than "tolp" apply rescaling.
meaningful only with ion_temperature = 'rescaling'
or with ion_velocities='change_step', where it specifies
the old timestep
```

[Back to Top]

nhpcl INTEGER

Default: 1

```
number of thermostats in the Nose-Hoover chain
currently maximum allowed is 4
```

[Back to Top]

nhptyp INTEGER

Default: 0

```
type of the "massive" Nose-Hoover chain thermostat
nhptyp=1 uses a NH chain per each atomic type
nhptyp=2 uses a NH chain per atom, this one is useful
for extremely rapid equipartitioning (equilibration is a
different beast)
nhptyp=3 together with nhgrp allows fine grained thermostat
control
NOTE: if using more than 1 thermostat per system there will
be a common thermostat added on top of them all, to disable
this common thermostat specify nhptyp=-X instead of nhptyp=X
```

[Back to Top]

nhgrp(i), i=1,ntyp INTEGER

Default: 0

```
specifies which thermostat group to use for given atomic type
when >0 assigns all the atoms in this type to thermostat
```

```
labeled nhgrp(i), when =0 each atom in the type gets its own
thermostat. Finally, when <0, then this atomic type will have
temperature "not controlled". Example: HCOOLi, with types H (1), C(2), O(3), Li(4);
setting nhgrp={2 2 0 -1} will add a common thermostat for both H & C,
one thermostat per each O (2 in total), and a non-updated thermostat
for Li which will effectively make temperature for Li "not controlled"
```

[Back to Top]

fnhscl(i), i=1,ntyp REAL

Default: (Nat_{total}-1)/Nat_{total}

```
these are the scaling factors to be used together with nhptyp=3 and nhgrp(i)
in order to take care of possible reduction in the degrees of freedom due to
constraints. Suppose that with the previous example HCOOLi, C-H bond is
constrained. Then, these 2 atoms will have 5 degrees of freedom in total instead
of 6, and one can set fnhscl={5/6 5/6 1. 1.}. This way the target kinetic energy
for H&C will become 6(kT/2)*5/6 = 5(kT/2). This option is to be used for
simulations with many constraints, such as rigid water with something else in there
```

[Back to Top]

ndega INTEGER

Default: 0

```
number of degrees of freedom used for temperature calculation
ndega <= 0 sets the number of degrees of freedom to
[3*nat-abs(ndega)], ndega > 0 is used as the target number
```

[Back to Top]

tranp(i), i=1,ntyp LOGICAL

Default: .false.

See: amprp

```
If .TRUE. randomize ionic positions for the
atomic type corresponding to the index.
```

[Back to Top]

amprp(i), i=1,ntyp REAL

Default: 0.D0

See: amprp

```
amplitude of the randomization for the atomic type corresponding
to the index i ( allowed values: 0.0 - 1.0 ).
meaningful only if " tranp(i) = .TRUE.".
```

[Back to Top]

greasp REAL

Default: 1.D0

```
same as "grease", for ionic damped dynamics.
```

[Back to Top]

Namelist: &CELL

input this namelist only if calculation = 'vc-relax', 'vc-cp', 'vc-cp-wf'

cell_parameters CHARACTER

```
'default'      : restart the simulation with cell parameters read
                 from the restart file or "celldm" if
                 "restart = 'from_scratch'"
'from_input'   : restart the simulation with cell parameters
                 from standard input.
```

```
( see the card 'CELL_PARAMETERS' )
```

[Back to Top]

cell_dynamics CHARACTER

Default: 'pr' if calculation = 'vc-md', 'vc'cp', 'vc-cp-wf'; 'damp-pr' if calculation = 'vc-relax'; 'none' otherwise

```
set how cell should be moved
'none'      : cell is kept fixed
'sd'        : steepest descent algorithm is used to optimise the
              cell
'damp-pr'   : damped dynamics is used to optimise the cell
              ( Parrinello-Rahman method ).
'pr'        : standard Verlet algorithm is used to propagate
              the cell ( Parrinello-Rahman method ).
```

[Back to Top]

cell_velocities CHARACTER

```
'zero'      : restart setting cell velocity to zero
'default'   : restart using cell velocity of the previous run
```

[Back to Top]

cell_damping REAL

Default: 0.1D0

```
damping frequency times delta t, optimal values could be
calculated with the formula :
         SQRT( 0.5 * LOG( ( E1 - E2 ) / ( E2 - E3 ) ) )
where E1, E2, E3 are successive values of the DFT total energy
in a steepest descent simulations.
meaningful only if " cell_dynamics = 'damp' "
```

[Back to Top]

press REAL

Default: 0.D0

```
Target pressure [KBar] in a variable-cell md or relaxation run.
```

[Back to Top]

wmass REAL

Default: 0.75*Tot_Mass/pi**2 for Parrinello-Rahman MD; 0.75*Tot_Mass/pi**2/Omega**(2/3) for Wentzcovitch MD

```
Fictitious cell mass [amu] for variable-cell simulations
(both 'vc-md' and 'vc-relax')
```

[Back to Top]

cell_factor REAL

Default: 1.2D0

```
Used in the construction of the pseudopotential tables.
It should exceed the maximum linear contraction of the
cell during a simulation.
```

[Back to Top]

cell_temperature CHARACTER

Default: 'not_controlled'

```
'nose'            : control cell temperature using Nose thermostat
                    see parameters "fnoseh" and "temph".
'rescaling'       : control cell temperature via velocities
                    rescaling.
'not_controlled'  : cell temperature is not controlled.
```

[Back to Top]

temph REAL

Default: 0.D0

```
value of the cell temperature (in ???) forced
by the temperature control.
meaningful only with " cell_temperature /= 'not_controlled' "
```

[Back to Top]

fnoseh REAL

Default: 1.D0

```
oscillation frequency of the nose thermostat (in terahertz)
meaningful only with " cell_temperature = 'nose' "
```

[Back to Top]

greash REAL

Default: 1.D0

```
same as "grease", for cell damped dynamics
```

[Back to Top]

cell_dofree CHARACTER

Default: 'all'

```
Select which of the cell parameters should be moved:

all     = all axis and angles are moved
x       = only the x component of axis 1 (v1_x) is moved
y       = only the y component of axis 2 (v2_y) is moved
z       = only the z component of axis 3 (v3_z) is moved
xy      = only v1_x and v2_y are moved
xz      = only v1_x and v3_z are moved
yz      = only v2_y and v3_z are moved
xyz     = only v1_x, v2_y, v3_z are moved
shape   = all axis and angles, keeping the volume fixed
2Dxy    = only x and y components are allowed to change
2Dshape = as above, keeping the area in xy plane fixed
volume  = isotropic variations of v1_x, v2_y, v3_z, keeping
          the shape fixed. Should be used only with ibrav=1.
```

[Back to Top]

Namelist: &PRESS_AI

input this namelist only when tabps = .true.

abivol LOGICAL

Default: .false.

```
.true. for finite pressure calculations
```

[Back to Top]

abisur LOGICAL

Default: .false.

```
.true. for finite surface tension calculations
```

[Back to Top]

P_ext REAL

Default: 0.D0

```
external pressure in GPa
```

[Back to Top]

pvar LOGICAL

Default: .false.

.true. for variable pressure calculations
pressure changes linearly with time:
Delta_P = (P_fin - P_in)/nstep

[Back to Top]

P_in REAL

Default: 0.D0

only if pvar = .true.
initial value of the external pressure (GPa)

[Back to Top]

P_fin REAL

Default: 0.D0

only if pvar = .true.
final value of the external pressure (GPa)

[Back to Top]

Surf_t REAL

Default: 0.D0

Surface tension (in a.u.; typical values 1.d-4 - 1.d-3)

[Back to Top]

rho_thr REAL

Default: 0.D0

threshold parameter which defines the electronic charge density
isosurface to compute the 'quantum' volume of the system
(typical values: 1.d-4 - 1.d-3)
(corresponds to alpha in PRL 94 145501 (2005))

[Back to Top]

dthr REAL

Default: 0.D0

thikness of the external skin of the electronic charge density
used to compute the 'quantum' surface
(typical values: 1.d-4 - 1.d-3; 50% to 100% of rho_thr)
(corresponds to Delta in PRL 94 145501 (2005))

[Back to Top]

Namelist: &WANNIER

only if calculation = 'cp-wf', 'vc-cp-wf'

Output files used by Wannier Function options are the following

```
      fort.21: Used only when calwf=5, contains the full list of g-vecs.
      fort.22: Used Only when calwf=5, contains the coeffs. corresponding
               to the g-vectors in fort.21
      fort.24: Used with calwf=3,contains the average spread
      fort.25: Used with calwf=3, contains the individual Wannier
               Function Spread of each state
      fort.26: Used with calwf=3, contains the wannier centers along a
               trajectory.
      fort.27: Used with calwf=3 and 4,  contains some general runtime
               information from ddyn, the subroutine that actually
```

```
                does the localization of the orbitals.
      fort.28: Used only if efield=.TRUE. , contains the polarization
                contribution to the total energy.

Also, The center of mass is fixed during the Molecular Dynamics.

BEWARE : THIS WILL ONLY WORK IF THE NUMBER OF PROCESSORS IS LESS THAN OR
         EQUAL TO THE NUMBER OF STATES.

Nota Bene 1:   For calwf = 5, wffort is not used. The
               Wannier/Wave(function) coefficients are written to unit 22
               and the corresponding g-vectors (basis vectors) are
               written to unit 21. This option gives the g-vecs and
               their coeffs. in reciprocal space, and the coeffs. are
               complex. You will have to convert them to real space
               if you want to plot them for visualization. calwf=1 gives
               the orbital densities in real space, and this is usually
               good enough for visualization.
```

wf_efield LOGICAL

Default: .false.

```
If dynamics will be done in the presence of a field
```

[Back to Top]

wf_switch LOGICAL

Default: .false.

```
Whether to turn on the field adiabatically (adiabatic switch)
if true, then nbeg is set to 0.
```

[Back to Top]

sw_len INTEGER

Default: 1

```
No. of iterations over which the field will be turned on
to its final value. Starting value is 0.0
If sw_len < 0, then it is set to 1.
If you want to just optimize structures on the presence of a
field, then you may set this to 1 and run a regular geometry
optimization.
```

[Back to Top]

efx0, efy0, efz0 REAL

See: 0.D0

```
Initial values of the field along x, y, and z directions
```

[Back to Top]

efx1, efy1, efz1 REAL

See: 0.D0

```
Final values of the field along x, y, and z directions
```

[Back to Top]

wfsd INTEGER

Default: 1

```
Localization algorithm for Wannier function calculation:
wfsd=1  Damped Dynamics
wfsd=2  Steepest-Descent / Conjugate-Gradient
wfsd=3  Jocobi Rotation
Remember, this is consistent with all the calwf options
as well as the tolw (see below).
Not a good idea to Wannier dynamics with this if you are
using restart='from_scratch' option, since the spreads
converge fast in the beginning and ortho goes bananas.
```

[Back to Top]

wfdt REAL

Default: 5.D0

The minimum step size to take in the SD/CG direction

[Back to Top]

maxwfdt REAL

Default: 0.3D0

The maximum step size to take in the SD/CG direction
The code calculates an optimum step size, but that may be
either too small (takes forever to converge) or too large
(code goes crazy) . This option keeps the step size between
wfdt and maxwfdt. In my experience 0.1 and 0.5 work quite
well. (but don't blame me if it doesn't work for you)

[Back to Top]

nit INTEGER

Default: 10

Number of iterations to do for Wannier convergence.

[Back to Top]

nsd INTEGER

Default: 10

Out of a total of NIT iterations, NSD will be Steepest-Descent
and (nit - nsd) will be Conjugate-Gradient.

[Back to Top]

wf_q REAL

Default: 1500.D0

Fictitious mass of the A matrix used for obtaining
maximally localized Wannier functions. The unitary
transformation matrix U is written as exp(A) where
A is a anti-hermitian matrix. The Damped-Dynamics is performed
in terms of the A matrix, and then U is computed from A.
Usually a value between 1500 and 2500 works fine, but should
be tested.

[Back to Top]

wf_friction REAL

Default: 0.3D0

Damping coefficient for Damped-Dynamics.

[Back to Top]

nsteps INTEGER

Default: 20

Number of Damped-Dynamics steps to be performed per CP
iteration.

[Back to Top]

tolw REAL

Default: 1.D-8

Convergence criterion for localization.

[Back to Top]

adapt LOGICAL

Default: .true.

Whether to adapt the damping parameter dynamically.

[Back to Top]

calwf INTEGER

Default: 3

Wannier Function Options, can be 1,2,3,4,5

1. Output the Wannier function density, nwf and wffort are used for this option. see below.
2. Output the Overlap matrix O_i,j=<w_i|exp{iGr}|w_j>. O is written to unit 38. For details on how O is constructed, see below.
3. Perform nsteps of Wannier dynamics per CP iteration, the orbitals are now Wannier Functions, not Kohn-Sham orbitals. This is a Unitary transformation of the occupied subspace and does not leave the CP Lagrangian invariant. Expectation values remain the same. So you will **NOT** have a constant of motion during the run. Don't freak out, its normal.
4. This option starts for the KS states and does 1 CP iteration and nsteps of Damped-Dynamics to generate maximally localized wannier functions. Its useful when you have the converged KS groundstate and want to get to the converged Wannier function groundstate in 1 CP Iteration.
5. This option is similar to calwf 1, except that the output is the Wannier function/wavefunction, and not the orbital density. See nwf below.

[Back to Top]

nwf INTEGER

Default: 0

This option is used with calwf 1 and calwf 5. with calwf=1, it tells the code how many Orbital densities are to be output. With calwf=5, set this to 1(i.e calwf=5 only writes one state during one run. so if you want 10 states, you have to run the code 10 times). With calwf=1, you can print many orbital densities in a single run.
See also the PLOT_WANNIER card for specifying the states to be printed.

[Back to Top]

wffort INTEGER

Default: 40

This tells the code where to dump the orbital densities. Used only with CALWF=1. for e.g. if you want to print 2 orbital densities, set calwf=1, nwf=2 and wffort to an appropriate number (e.g. 40) then the first orbital density will be output to fort.40, the second to fort.41 and so on. Note that in the current implementation, the following units are used 21,22,24,25,26,27,28,38,39,77,78 and whatever you define as ndr and ndw. so use number other than these.

[Back to Top]

writev LOGICAL

Default: .false.

Output the charge density (g-space) and the list of g-vectors
This is useful if you want to reconstruct the electrostatic potential using the Poisson equation. If .TRUE. then the code will output the g-space charge density and the list if G-vectors, and STOP.
Charge density is written to : CH_DEN_G_PARA.ispin (1 or 2 depending on the number of spin types) or CH_DEN_G_SERL.ispin depending on if the code is being run in parallel or serial
G-vectors are written to G_PARA or G_SERL.

[Back to Top]

exx_neigh INTEGER

Default: 60

An initial guess on the maximum number of neighboring (overlapping) MLWFs.

[Back to Top]

exx_dis_cutoff REAL

Default: 8.0

Radial cutoff distance (in bohr) for including overlapping MLWF pairs
in EXX calculations.
See J. Chem. Theory Comput. 16, 3757â3785 (2020).

[Back to Top]

exx_poisson_eps REAL

Default: 1.0D-6

Poisson solver convergence criterion during computation of the EXX potential.

[Back to Top]

exx_use_cube_domain LOGICAL

Default: .false.

Use cubic instead of spherical subdomains as local supports during computation
of the EXX potential. If set to .TRUE., the spherical domain
radii (exx_ps_rcut_self, exx_ps_rcut_pair, exx_me_rcut_self, exx_me_rcut_pair)
will be treated as half of the side length of the cubic subdomain.

[Back to Top]

exx_ps_rcut_self REAL

Default: 6.0

See: exx_use_cube_domain

Radial cutoff distance (in bohr) to compute the self EXX energy.
This distance determines the radius of the Poisson sphere centered at
a given MLWF center, and should be large enough to cover
the majority of the MLWF charge density.
See J. Chem. Theory Comput. 16, 3757â3785 (2020).

[Back to Top]

exx_ps_rcut_pair REAL

Default: 5.0

See: exx_use_cube_domain

Radial cutoff distance (in bohr) to compute the pair EXX energy.
This distance determines the radius of the Poisson sphere centered at
the midpoint of two overlapping MLWFs, and should be
large enough to cover the majority of the MLWF product density.
This parameter can generally be chosen as smaller than exx_ps_rcut_self.
See J. Chem. Theory Comput. 16, 3757â3785 (2020).

[Back to Top]

exx_me_rcut_self REAL

Default: 10.0

See: exx_use_cube_domain

Radial cutoff distance (in bohr) for the multipole-expansion sphere
centered at a given MLWF center.
The far-field self EXX potential in this sphere is generated with a
multipole expansion of the MLWF charge density.
This parameter must be larger than exx_ps_rcut_self by at least 3
real-space grid point spacings.
See J. Chem. Theory Comput. 16, 3757â3785 (2020).

[Back to Top]

exx_me_rcut_pair REAL

Default: 7.0

See: exx_use_cube_domain

Radial cutoff distance (in bohr) for the multipole-expansion sphere
centered at the midpoint of two overlapping MLWFs.
The far-field pair EXX potential in this sphere is generated with
a multipole expansion of the MLWF product density.
This parameter must be larger than exx_ps_rcut_pair by at least 3
real-space grid point spacings. Also, this parameter can generally
be chosen as smaller than exx_me_rcut_self.
See J. Chem. Theory Comput. 16, 3757â3785 (2020).

[Back to Top]

Card: ATOMIC_SPECIES

Syntax:

ATOMIC_SPECIES
X(1) *Mass_X(1)* *PseudoPot_X(1)*
X(2) *Mass_X(2)* *PseudoPot_X(2)*
...
X(ntyp) *Mass_X(ntyp)* *PseudoPot_X(ntyp)*

Description of items:

X CHARACTER

label of the atom. Acceptable syntax:
chemical symbol X (1 or 2 characters, case-insensitive)
or chemical symbol plus a number or a letter, as in
"Xn" (e.g. Fe1) or "X_*" or "X-*" (e.g. C1, C_h;
max total length cannot exceed 3 characters)

[Back to Top]

Mass_X REAL

mass of the atomic species [amu: mass of C = 12]
not used if calculation='scf', 'nscf', 'bands'

[Back to Top]

PseudoPot_X CHARACTER

File containing PP for this species.

The pseudopotential file is assumed to be in the new UPF format.
If it doesn't work, the pseudopotential format is determined by
the file name:

*.vdb or *.van Vanderbilt US pseudopotential code
*.RRKJ3 Andrea Dal Corso's code (old format)
none of the above old PWscf norm-conserving format

[Back to Top]

Card: ATOMIC_POSITIONS { alat | bohr | angstrom | crystal }

IF *calculation == 'bands' OR calculation == 'nscf'* :

Specified atomic positions will be IGNORED and those from the
previous scf calculation will be used instead !!!

ELSEIF :

Syntax:

ATOMIC_POSITIONS { alat | bohr | angstrom | crystal }

X(1) *x(1)* *y(1)* *z(1)* { *if_pos(1)(1)* *if_pos(2)(1)* *if_pos(3)(1)* }
X(2) *x(2)* *y(2)* *z(2)* { *if_pos(1)(2)* *if_pos(2)(2)* *if_pos(3)(2)* }
...
X(nat) *x(nat)* *y(nat)* *z(nat)* { *if_pos(1)(nat)* *if_pos(2)(nat)* *if_pos(3)(nat)* }

Description of items:

Card's options: **alat | bohr | angstrom | crystal**

Default: (DEPRECATED) bohr

```
alat    : atomic positions are in cartesian coordinates,
          in units of the lattice parameter (either
          celldm(1) or A).

bohr    : atomic positions are in cartesian coordinate,
          in atomic units (i.e. Bohr).
          If no option is specified, 'bohr' is assumed;
          not specifying units is DEPRECATED and will no
          longer be allowed in the future

angstrom: atomic positions are in cartesian coordinates,
          in Angstrom

crystal : atomic positions are in crystal coordinates, i.e.
          in relative coordinates of the primitive lattice
          vectors as defined either in card CELL_PARAMETERS
          or via the ibrav + celldm / a,b,c... variables
```

[Back to Top]

X CHARACTER

```
label of the atom as specified in ATOMIC_SPECIES
```

[Back to Top]

x, y, z REAL

```
atomic positions
```

[Back to Top]

if_pos(1), if_pos(2), if_pos(3) INTEGER

Default: 1

```
component i of the force for this atom is multiplied by if_pos(i),
which must be either 0 or 1.  Used to keep selected atoms and/or
selected components fixed in MD dynamics or
structural optimization run.
```

[Back to Top]

Card: ATOMIC_VELOCITIES

Optional card, reads velocities from standard input

```
when starting with ion_velocities = "from_input" it is convenient
to perform a few steps (~5-10) with a small time step (0.5 a.u.).
The velocities must be expressed using the same length units
```

indicated in the card ATOMIC_POSITIONS, divided by time
in atomic units.

Syntax:

ATOMIC_VELOCITIES

V(1) *vx(1)* *vy(1)* *vz(1)*
V(2) *vx(2)* *vy(2)* *vz(2)*
...
V(nat) *vx(nat)* *vy(nat)* *vz(nat)*

Description of items:

V CHARACTER

label of the atom as specified in ATOMIC_SPECIES

[Back to Top]

vx, vy, vz REAL

atomic velocities along *x*, *y* and *z* direction

[Back to Top]

Card: CELL_PARAMETERS { bohr | angstrom | alat }

Optional card, needed only if ibrav = 0 is specified, ignored otherwise !

Syntax:

CELL_PARAMETERS { bohr | angstrom | alat }

v1(1) *v1(2)* *v1(3)*
v2(1) *v2(2)* *v2(3)*
v3(1) *v3(2)* *v3(3)*

Description of items:

Card's options: **bohr | angstrom | alat**

'bohr'/'angstrom': lattice vectors in bohr radii / angstrom.
'alat' / nothing specified: lattice vectors in units or the
lattice parameter (either celldm(1) or a). Not specifing
units is DEPRECATED and will not be allowed in the future.
If nothing specified and no lattice parameter specified,
'bohr' is assumed - DEPRECATED, will no longer be allowed

[Back to Top]

v1, v2, v3 REAL

Crystal lattice vectors:
v1(1) v1(2) v1(3) ... 1st lattice vector
v2(1) v2(2) v2(3) ... 2nd lattice vector
v3(1) v3(2) v3(3) ... 3rd lattice vector

[Back to Top]

Card: REF_CELL_PARAMETERS { bohr | angstrom }

Optional card, needed only if one wants to do variable cell calculations accurately. The reference cell generates additional buffer planewaves.

Syntax:

REF_CELL_PARAMETERS { bohr | angstrom }

v1(1) *v1(2)* *v1(3)*
v2(1) *v2(2)* *v2(3)*
v3(1) *v3(2)* *v3(3)*

Description of items:

Card's options: **bohr | angstrom**

```
bohr / angstrom: reference cell parameters in bohr radii / angstrom.

To mimic a constant effective planewave kinetic energy (ecfixed) during a
variable-cell calculation, the specified reference cell has to be large enough
such that the individual cell vector lengths of the fluctuating cell do not
exceed the corresponding reference lattice vector lengths during the entire
calculation. The cost of the calculation will increase with the increasing
size of the reference cell. The user must test for the proper reference cell
parameters.

The reference cell parameters should be used in conjunction with q2sigma,
qcutz, and ecfixed. See q2sigma for more information about mimicking constant
effective planewave kinetic energy (ecfixed) during variable-cell calculations.

The reference cell parameters should be chosen as an isotropic scaling of the
initial cell of the system. This means that the reference cell should have
the same shape as the initial simulatoin cell. The reference cell parameters should
NOT be changed throughout a given simulatoin. Typically, 2%-10% scaling of
the unit cell vectors are sufficient. However, the cell fluctuations depend on
the system and the thermodynamic conditions. So again user must test for the proper
choice of reference cell parameters.
```

[Back to Top]

v1, v2, v3 REAL

```
REF_CELL_PARAMETERS { bohr | angstrom }
v1(1)  v1(2)  v1(3)    ... 1st reference lattice vector
v2(1)  v2(2)  v2(3)    ... 2nd reference lattice vector
v3(1)  v3(2)  v3(3)    ... 3rd reference lattice vector
```

[Back to Top]

Card: CONSTRAINTS

Optional card, used for constrained dynamics or constrained optimisations

```
When this card is present the SHAKE algorithm is automatically used.
```

Syntax:

CONSTRAINTS
nconstr { *constr_tol* }
constr_type(1) *constr(1)(1)* *constr(2)(1)* [*constr(3)(1)* *constr(4)(1)*] { *constr_target(1)* }
constr_type(2) *constr(1)(2)* *constr(2)(2)* [*constr(3)(2)* *constr(4)(2)*] { *constr_target(2)* }
...
constr_type(nconstr) *constr(1)(nconstr)* *constr(2)(nconstr)* [*constr(3)(nconstr)* *constr(4)(nconstr)*] { *constr_target(nconstr)* }

Description of items:

nconstr INTEGER

```
Number of constraints.
```

[Back to Top]

constr_tol REAL

```
Tolerance for keeping the constraints satisfied.
```

[Back to Top]

constr_type CHARACTER

```
Type of constrain :

'type_coord'      : constraint on global coordination-number, i.e. the
                    average number of atoms of type B surrounding the
                    atoms of type A. The coordination is defined by
                    using a Fermi-Dirac.
                    (four indexes must be specified).

'atom_coord'      : constraint on local coordination-number, i.e. the
                    average number of atoms of type A surrounding a
                    specific atom. The coordination is defined by
                    using a Fermi-Dirac.
                    (four indexes must be specified).

'distance'        : constraint on interatomic distance
                    (two atom indexes must be specified).

'planar_angle'    : constraint on planar angle
                    (three atom indexes must be specified).

'torsional_angle' : constraint on torsional angle
                    (four atom indexes must be specified).

'bennett_proj'    : constraint on the projection onto a given direction
                    of the vector defined by the position of one atom
                    minus the center of mass of the others.
                    ( Ch.H. Bennett in Diffusion in Solids, Recent
                      Developments, Ed. by A.S. Nowick and J.J. Burton,
                      New York 1975 ).
```

[Back to Top]

constr(1), constr(2), constr(3), constr(4)

```
                        These variables have different meanings
                        for different constraint types:

                      'type_coord' : constr(1) is the first index of the
                                     atomic type involved
                                     constr(2) is the second index of the
                                     atomic type involved
                                     constr(3) is the cut-off radius for
                                     estimating the coordination
                                     constr(4) is a smoothing parameter

                      'atom_coord' : constr(1) is the atom index of the
                                     atom with constrained coordination
                                     constr(2) is the index of the atomic
                                     type involved in the coordination
                                     constr(3) is the cut-off radius for
                                     estimating the coordination
                                     constr(4) is a smoothing parameter

                        'distance' : atoms indices object of the
                                     constraint, as they appear in
                                     the 'ATOMIC_POSITION' CARD

'planar_angle', 'torsional_angle' : atoms indices object of the
                                     constraint, as they appear in the
                                     'ATOMIC_POSITION' CARD (beware the
                                     order)

                    'bennett_proj' : constr(1) is the index of the atom
                                     whose position is constrained.
                                     constr(2:4) are the three coordinates
                                     of the vector that specifies the
                                     constraint direction.
```

[Back to Top]

constr_target REAL

Target for the constrain (angles are specified in degrees).
This variable is optional.

[Back to Top]

Card: OCCUPATIONS

Optional card, used only if occupations = 'from_input', ignored otherwise !

Syntax:

OCCUPATIONS
f_inp1(1) *f_inp1(2)* ... *f_inp1(nbnd)*
[*f_inp2(1)* *f_inp2(2)* ... *f_inp2(nbnd)*]

Description of items:

f_inp1 REAL

Occupations of individual states (MAX 10 PER LINE).
For spin-polarized calculations, these are majority spin states.

[Back to Top]

f_inp2 REAL

Occupations of minority spin states (MAX 10 PER LINE)
To be specified only for spin-polarized calculations.

[Back to Top]

Card: ATOMIC_FORCES

Optional card used to specify external forces acting on atoms

Syntax:

ATOMIC_FORCES
X(1) *fx(1)* *fy(1)* *fz(1)*
X(2) *fx(2)* *fy(2)* *fz(2)*
...
X(nat) *fx(nat)* *fy(nat)* *fz(nat)*

Description of items:

X CHARACTER

label of the atom as specified in ATOMIC_SPECIES

[Back to Top]

fx, fy, fz REAL

external force on atom X (cartesian components, Ha/a.u. units)

[Back to Top]

Card: PLOT_WANNIER

Optional card, indices of the states that have to be printed (only for calf=1 and calf=5).

Syntax:

PLOT_WANNIER
iwf(1)
iwf(2)
. . .
iwf(nwf)

Description of items:

iwf	INTEGER
These are the indices of the states that you want to output. Also used with calwf = 1 and 5. If calwf = 1, then you need nwf indices here (each in a new line). If CALWF=5, then just one index in needed.	

[Back to Top]

AUTOPILOT

Syntax of this supercard is the following:

```
AUTOPILOT
  ... content of the supercard here ...
ENDRULES
```

and the content is:

Optional card, changes some variables on the fly of the calculation.

Notice that the rules has to be ordered in with time step and the **AUTOPILOT** card has to be terminated with the **ENDRULES** keyword.

To set up a rule, one can add the scheduled steps with **on_step** and separate the corresponding change in parameters with a column.

A simple example:

```
AUTOPILOT
    on_step =  31  : dt               = 5.0
    on_step =  91  : iprint           = 100
    on_step =  91  : isave            = 100
    on_step = 191  : ion_dynamics     = 'damp'
    on_step = 191  : electron_damping = 0.00
    on_step = 691  : ion_temperature  = 'nose'
    on_step = 691  : tempw            = 150.0
ENDRULES
```

ENDRULES